

SMART HOME LIGHTING SYSTEM

Voice Control & Motion Detection Using ESP32 & Blynk IoT

A Project Report Submitted by

GROUP 15

Name	Student ID
Cyril Osah Martey	7329123
Cudjoe Bless	7321323
Fraizer Akwasi Clement	7324123
Gunab Ebenezer	7324923
Benard Fosu Marfo	7329023
Nyamekye Busumbru	7321123
Grace Sika Konadu	7326823
Millicent Djabatey	7322423
Juanita Da Pilma	7321523

Department of Electrical & Electronic Engineering
IoT Systems Design — 2026

1. Introduction

The rapid advancement of the Internet of Things (IoT) has opened transformative possibilities for home automation, enabling everyday devices to communicate intelligently over wireless networks. Smart lighting, in particular, represents one of the most impactful areas of IoT application — reducing energy waste, enhancing comfort, and improving security. This project, undertaken by Group 15, embraces that vision by designing and implementing a fully functional Smart Home Lighting System that combines automated motion sensing with remote voice-controlled operation.

The system is built around the ESP32 DOIT DevKit V1, a powerful and Wi-Fi-enabled microcontroller, and leverages the Blynk IoT cloud platform to enable real-time remote control via a mobile application. A PIR (Passive Infrared) motion sensor monitors physical activity within a room and automatically activates the lighting system upon detecting movement. Simultaneously, users can override or toggle the lights manually through voice commands or switch widgets on the Blynk mobile application — providing a dual-mode control experience that is both intelligent and user-friendly.

This report comprehensively documents the journey from concept to working prototype: the tools and components selected, the simulation process, the physical build, the code logic, the challenges encountered and resolved, and the final results achieved. It represents the collective effort of nine students who contributed across different phases of the project.

2. Project Overview

The Smart Home Lighting System is an IoT-based solution designed to automate and remotely control home lighting through two complementary mechanisms. The project addresses a common challenge in modern homes: the need for a lighting system that is both reactive to occupancy and accessible from anywhere via a smartphone.

The core objectives of the project were as follows:

- To build an automated lighting system that responds to physical motion using a PIR sensor.

- To integrate remote control capabilities through the Blynk mobile application, allowing users to toggle lighting via a voice widget or switch.
- To display real-time system status on a 16×2 I2C LCD screen mounted as part of the circuit.
- To simulate the design virtually before assembling the physical prototype, ensuring reliability.
- To identify, debug, and resolve all software and hardware issues encountered during implementation.

The system successfully integrates four key functional areas: Smart Control (remote management from anywhere via the Blynk app), Motion Detection (the PIR sensor auto-triggers the LED upon detecting movement), Voice Commands (turning lights ON or OFF using the Blynk voice widget), and LCD Display (showing real-time status updates on the 16×2 I2C screen). Together, these features create a robust and practical smart lighting prototype.

3. Tools and Components Used

The project made use of a carefully selected combination of hardware components and software platforms to achieve its goals. Each element played a distinct and critical role in the overall system.

3.1 Hardware Components

Component	Role in the Project
ESP32 DOIT DevKit V1	Main microcontroller — handles Wi-Fi connectivity, executes the program, and interfaces with all sensors and output devices.
HC-SR501 PIR Sensor	Detects motion by sensing infrared radiation from moving bodies. Triggers the LED when movement is detected.
16×2 I2C LCD Display (0x27)	Outputs real-time system status messages such as 'Motion Detected' or 'System Ready' to the user.
LED + 220 Ohm Resistor	Serves as the light indicator. The resistor protects the LED from excess current.
Breadboard & Jumper Wires	Used for prototyping — allows components to be connected without soldering.
USB Cable	Provides power to the ESP32 and enables programming from the Arduino IDE.
iPhone Hotspot	Supplies the Wi-Fi connectivity required for the ESP32 to connect to the Blynk cloud server.

3.2 Software and Platforms

The software stack was equally important to the project's success. The following tools were used throughout development:

- **Arduino IDE 2.3.4** — The primary development environment used to write, compile, and upload the C++ code to the ESP32 microcontroller.
- **Wokwi Simulator** — A free online IoT simulator that allowed the group to build and test the entire circuit virtually before touching physical hardware.
- **Blynk IoT Platform** — The cloud-based platform that hosts the mobile application, handles voice control input, and manages Virtual Pin (V1) communication with the ESP32.
- **LiquidCrystal_I2C Library** — Controls the I2C LCD display, allowing the ESP32 to send messages to the screen with minimal wiring.
- **BlynkSimpleEsp32.h** — Provides the Wi-Fi and Blynk communication functions.
- **WiFi.h / WiFiClient.h** — Core network libraries for managing the Wi-Fi connection on the ESP32.
- **Wire.h** — Handles I2C communication protocol between the ESP32 and the LCD display.

4. Wokwi Simulation

4.1 What is Wokwi?

Wokwi is a free, browser-based IoT and embedded systems simulator that allows developers and students to design, program, and test microcontroller projects entirely in a virtual environment — without needing any physical hardware. It supports a wide range of components including the ESP32, Arduino boards, sensors, displays, and various output devices.

For Group 15, Wokwi provided the ideal first step: a risk-free environment to validate the system logic before committing to physical assembly. This approach proved invaluable in identifying code-level issues early and confirming that the overall design was sound.

4.2 Components Simulated

The following components were successfully simulated in the Wokwi environment:

- **ESP32 DevKit V1** — The main microcontroller running the full program.

- HC-SR501 PIR Sensor — Simulated motion detection by toggling the sensor state within the simulator.
- 16×2 I2C LCD Display — Displayed real-time status messages exactly as expected.
- LED with 220 Ohm resistor — Served as the visual light indicator.
- Blynk Virtual Pin V1 — Simulated the voice/switch control input from the Blynk mobile app.

4.3 Simulation Results

All functions were verified successfully during the Wokwi simulation phase. Motion detection triggered the LED correctly and updated the LCD display. Voice and switch control through the Blynk Virtual Pin V1 toggled the LED ON and OFF as designed. The auto shut-off logic (10-second timer) functioned as expected. This successful simulation gave the group the confidence to proceed with the physical build.

5. Code Explanation

5.1 Simulation Code

The simulation code was structured into five logical sections, each performing a specific function within the overall system:

Section 1 — Blynk Configuration (Lines 1–5): The code begins by defining the Blynk Template ID, Template Name, and Authentication Token. These three identifiers uniquely link the physical device to its corresponding project on the Blynk cloud server, enabling secure and authenticated communication.

Section 2 — Library Imports and Pin Setup (Lines 7–26): All necessary libraries are imported, including the WiFi, Blynk, and LCD libraries. The LED is declared on GPIO18, the PIR sensor is assigned to GPIO13, and the LCD is configured to operate at I2C address 0x27.

Section 3 — Setup Function (Lines 29–65): This function runs once at startup. It initialises the LCD screen, establishes a Wi-Fi connection (using Wokwi-GUEST credentials for the simulation environment), and connects the device to the Blynk cloud server.

Section 4 — Voice Control Handler (Lines 68–74): A dedicated function listens to Virtual Pin V1 from the Blynk app. When the switch widget is toggled by the user, the function reads the incoming integer value and sets the LED state accordingly — 1 for ON and 0 for OFF.

Section 5 — Motion Detection Loop (Lines 76–110): The main loop reads the PIR sensor state every 100 milliseconds. When the sensor output goes HIGH (indicating motion), the LED is switched ON, the LCD displays 'Motion Detected', and a 10-second countdown begins. After the timer expires, the LED turns OFF automatically.

5.2 Physical Implementation Code

When transitioning to the physical hardware, the simulation code required several targeted modifications. These changes ensured compatibility with real-world conditions that differ from the controlled simulation environment, and are described in detail in the following section.

6. Physical Implementation

6.1 Circuit Connections

The physical circuit was assembled on a breadboard according to the following pin connections:

Component / Pin	Connected To
LED (via 220 Ohm Resistor)	GPIO 18
PIR Sensor OUT	GPIO 13
PIR Sensor VCC	5V
PIR Sensor GND	GND
LCD SDA	GPIO 21
LCD SCL	GPIO 22
LCD VCC	V5 (5V pin)
LCD GND	GND

6.2 Key Differences from Simulation

Three important differences distinguish the physical implementation code from the simulation version. First, the Wi-Fi credentials were updated from the Wokwi guest network to the real iPhone hotspot. Second, the `lcd.init()` function was called twice — a requirement unique to ESP32 hardware to properly reset the I2C LCD and prevent garbled characters from appearing at startup. Third, the `Wire.begin(21, 22)` function was explicitly called to declare the I2C pins, which is required for correct communication in the physical environment.

7. Code Fixes and Debugging

The transition from simulation to physical hardware revealed four distinct bugs that required systematic debugging and resolution. Each fix is detailed below.

Fix 1: Wi-Fi Credentials

In the simulation, the ESP32 connected to the virtual Wokwi-GUEST network using `WiFi.begin("Wokwi-GUEST", "", 6)`. This network does not exist in the real world. For the physical build, the credentials were replaced with the actual iPhone hotspot name and password, stored in char arrays `ssid[]` and `pass[]`. Without this fix, the ESP32 would enter an infinite connection loop and never reach the Blynk server.

Fix 2: LCD Double Initialisation

When the physical ESP32 first started up, the LCD display consistently showed garbled or random characters instead of the expected startup message. This was traced to the ESP32's I2C initialisation behaviour, which requires the `lcd.init()` function to be called twice in succession to fully reset the display driver. Calling it only once — as in the simulation — is insufficient for the hardware. Adding the second call resolved the issue completely.

Fix 3: Motion Detection Logic Bug

A subtle but critical logical error was discovered in the motion detection condition. The original code checked if `(!ledState)` before activating the LED — using the variable that tracks the voice-control

state. This meant that if the user had previously turned off the LED via the Blynk app, the motion detection would be blocked from turning it back on. The fix replaced `!ledState` with `!motionDetected` — a dedicated boolean flag that correctly tracks whether the motion-triggered light cycle is already active, completely independent of the voice control state.

Fix 4: 10-Second Auto OFF Logic

In the original code, the LED auto-off logic was placed inside an else if block that checked for a LOW PIR state after motion was detected. In practice, the PIR sensor's output fluctuates briefly during and after detection, causing the condition to be missed and the LED to remain ON indefinitely. The fix relocated the OFF command — `delay(10000)` followed by `digitalWrite(ledPin, LOW)` and `motionDetected = false` — directly inside the motion detection block, guaranteeing the 10-second timer and auto-off would always execute after every valid motion trigger.

8. How the System Works

8.1 Motion Detection Path

The motion-triggered operation follows a clear and sequential flow. When the ESP32 is powered on, it boots up and immediately attempts to connect to the configured Wi-Fi network. Once connectivity is established, it connects to the Blynk cloud server. From this point, the main program loop runs continuously, polling the PIR sensor every 100 milliseconds. When the sensor detects movement, it outputs a HIGH signal to GPIO 13. The ESP32 reads this signal, turns the LED ON via GPIO 18, and updates the LCD to display 'Motion Detected'. A 10-second timer begins, after which the LED is turned OFF automatically and the system returns to standby, ready for the next detection event.

8.2 Voice Control Path

The Blynk voice/switch control path operates independently but in parallel with the motion detection logic. The user opens the Blynk mobile application and interacts with the Switch Widget configured on Virtual Pin V1. When the widget is toggled, the command is transmitted over Wi-Fi to the Blynk cloud, which forwards it to the ESP32. The `BLYNK_WRITE(V1)` handler function on the

microcontroller receives the signal, reads the incoming integer value, and immediately updates the LED state — turning it ON (value 1) or OFF (value 0). This response is nearly instantaneous and works as long as both the device and the user's phone are connected to the internet.

9. Challenges and Solutions

No hardware project is without its difficulties. Group 15 encountered four significant challenges during the course of the project. Each was systematically identified and resolved through methodical troubleshooting.

Challenge 1: Wi-Fi Connection Failure (Status Code 6)

During the initial upload to the physical hardware, the ESP32 repeatedly failed to connect to the Wi-Fi network, reporting a connection status code of 6 — indicating a Wi-Fi connection failure. After investigation, the issue was traced to the iPhone's Personal Hotspot settings. iPhones, by default, can restrict connections from certain devices. The solution was twofold: the hotspot name was simplified to 'Gunab' (a short, clean SSID), and the 'Maximize Compatibility' option was enabled in the iPhone's Personal Hotspot settings. This allowed the ESP32 to connect successfully and reliably.

Challenge 2: LCD Showing Garbage Characters

On startup, the 16×2 LCD consistently displayed random characters, symbols, or blank lines instead of the intended messages. This is a known quirk of using I2C LCD displays with the ESP32: the I2C bus initialisation can be incomplete after a single `lcd.init()` call. The solution was to call `lcd.init()` twice in the setup function, which forces a full hardware reset of the LCD driver and clears any corrupt display state.

Challenge 3: LED Not Turning OFF After 10 Seconds

During initial physical testing, the LED would activate upon motion detection but never turn off — even after waiting well beyond the intended 10-second period. Root-cause analysis revealed a logic conflict: the auto-off condition referenced the `ledState` variable (used by the voice control), which was

already set to LOW. This prevented the motion-triggered OFF logic from ever executing. Replacing `ledState` with a dedicated `motionDetected` flag, and relocating the OFF command inside the detection block, permanently resolved the issue.

Challenge 4: Compilation exit status 1 Error

During code upload to the physical board, the Arduino IDE threw a compilation exit status 1 error with no descriptive message. By enabling verbose output in the IDE's preferences, the hidden error was revealed: the code still contained the Wokwi-GUEST Wi-Fi credentials that are incompatible with the real hardware. Once these were replaced with the actual hotspot credentials, the compilation and upload succeeded without further issues.

10. Results and Demonstration

The project achieved complete success across all three evaluation fronts: simulation, physical build, and cloud connectivity.

10.1 Simulation Results

All features of the smart lighting system were verified on the Wokwi simulator before any physical assembly. Motion detection responded correctly to sensor input, the LED turned ON and OFF as expected, the LCD displayed accurate status messages, and voice control via Virtual Pin V1 toggled the LED successfully. The simulation returned 100% functional results across every test case.

10.2 Physical Build Results

The physical prototype was assembled on a breadboard with the ESP32 DOIT DevKit V1 at its centre, connected to the PIR sensor, the 16×2 I2C LCD, and the LED circuit. Following the debugging fixes described in Section 7, all components responded correctly to the uploaded code. The PIR sensor accurately detected motion, the LED toggled as expected, the LCD displayed real-time status updates, and the 10-second auto-off timer operated precisely.

10.3 Blynk App Connectivity

The device was confirmed to be online and visible within the Blynk IoT dashboard. The Switch Widget on Virtual Pin V1 successfully toggled the LED ON and OFF with near-instant response times. Remote control from the mobile application worked seamlessly over the iPhone hotspot, demonstrating the end-to-end IoT connectivity of the system.

11. Conclusion

This project successfully demonstrated the design and implementation of a fully functional Smart Home Lighting System using an ESP32 microcontroller, a PIR motion sensor, a 16×2 I2C LCD display, and the Blynk IoT cloud platform. The system combines physical automation with remote digital control — a hallmark of modern IoT design.

The project was executed in a rigorous, phased manner: the circuit and code were first validated entirely in the Wokwi simulation environment, then transferred to physical hardware with targeted code fixes. Every bug encountered — from Wi-Fi connection failures to LCD display corruption and logic errors — was systematically identified and resolved. The final prototype met all intended objectives: motion-triggered lighting with a 10-second auto-off timer, remote voice/switch control via the Blynk app, and real-time LCD status feedback.

Beyond the technical achievement, this project provided Group 15 with hands-on experience in embedded systems programming, IoT cloud integration, hardware debugging, and collaborative engineering. The skills developed — from writing interrupt-driven sensor code to configuring cloud-connected devices — are directly applicable to real-world IoT development and reflect the growing importance of smart technology in everyday life.

Group 15 takes pride in the outcome of this project and extends appreciation to all members who contributed their expertise, time, and dedication across every phase of its development.

— Group 15 | *Smart Home Lighting System* | *ESP32 & Blynk IoT Project* —